

# MCA Semester III

## Advance Java Programming (Spring Boot)

### Practical Assignment

#### Index of Practicals

| No. | Practical Title                                                                                  | Textbook Chapter                     |
|-----|--------------------------------------------------------------------------------------------------|--------------------------------------|
| 1   | Introduction to JPA – Entity Class and Persisting a Single Entity                                | Ch 5: Working with Spring Data JPA   |
| 2   | Spring Data Repository Hierarchy – CrudRepository vs PagingAndSortingRepository vs JpaRepository | Ch 5: Working with Spring Data JPA   |
| 3   | Custom Query Methods – findBy, countBy and @Query (JPQL)                                         | Ch 5: Working with Spring Data JPA   |
| 4   | One-to-One Mapping – Three Worked Examples                                                       | Ch 5: Working with Spring Data JPA   |
| 5   | One-to-Many / Many-to-One Mapping – Department and Employee                                      | Ch 5: Working with Spring Data JPA   |
| 6   | Many-to-Many Mapping – Student and Course                                                        | Ch 5: Working with Spring Data JPA   |
| 7   | Transaction Management with @Transactional – Bank Fund Transfer                                  | Ch 5: Working with Spring Data JPA   |
| 8   | Pagination and Sorting with Spring Data JPA                                                      | Ch 5: Working with Spring Data JPA   |
| 9   | Building a RESTful Microservice using @RestController and ResponseEntity                         | Ch 6: Building RESTful Microservices |
| 10  | Layered Microservice Architecture – RestController + Service + JPA Repository                    | Ch 6: Building RESTful Microservices |
| 11  | Handling Request Data – @PathVariable, @RequestParam and @RequestBody                            | Ch 6: Building RESTful Microservices |
| 12  | DTO Pattern – Separating the Entity from the API Contract                                        | Ch 6: Building RESTful Microservices |
| 13  | Request Validation using Bean Validation (@Valid) with RestController                            | Ch 6: Building RESTful Microservices |
| 14  | Global Exception Handling using @ControllerAdvice and @ExceptionHandler                          | Ch 6: Building RESTful Microservices |
| 15  | Securing REST API with HTTP Basic Authentication (Spring Security)                               | Ch 7: Securing a Web Application     |
| 16  | User Registration and Login with BCrypt Password Encoding                                        | Ch 7: Securing a Web Application     |
| 17  | Role-Based Access Control (RBAC) with Spring Security                                            | Ch 7: Securing a Web Application     |
| 18  | Securing REST API as an OAuth2 Resource Server with JWT                                          | Ch 7: Securing a Web Application     |
| 19  | OAuth2 Login with a Third-Party Provider (Google/GitHub) and Accessing a Secured REST API        | Ch 7: Securing a Web Application     |
| 20  | Cross-Origin Resource Sharing (CORS) Configuration for Secured REST APIs                         | Ch 7: Securing a Web Application     |

## Chapter 5 — Working with Spring Data JPA

### Practical No. 1: Introduction to JPA – Entity Class and Persisting a Single Entity

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

#### Objective:

To understand the requirements of a JPA entity class (no-arg constructor, @Entity, @Id, @GeneratedValue), configure a relational datasource in application.properties, and persist/retrieve an entity using Spring Data JPA's CrudRepository.

#### Class Name(s) with Attributes:

##### *Employee* (@Entity)

- empId : int (@Id @GeneratedValue)
- empName : String
- department : String
- salary : double

##### *EmployeeRepository* (interface extends CrudRepository<Employee, Integer>)

- Inherited: save(), findAll(), findById(), deleteById()

##### *EmployeeRestController* (@RestController)

- employeeRepository : EmployeeRepository

#### Step-by-Step Task Instructions:

1. Add spring-boot-starter-data-jpa and a database driver (MySQL/H2) dependency in pom.xml.
2. Configure spring.datasource.url, username, password and spring.jpa.hibernate.ddl-auto=update in application.properties.
3. Create the Employee entity annotated with @Entity, @Id, and @GeneratedValue(strategy = GenerationType.IDENTITY).
4. Create EmployeeRepository extending CrudRepository<Employee, Integer> (the base Spring Data repository interface introduced in this chapter).
5. Create EmployeeRestController with POST /api/employees (save) and GET /api/employees (findAll).
6. Add GET /api/employees/{id} (findById) to fetch a single employee, PUT /api/employees/{id} to update an existing employee (fetch, modify fields, save again), and DELETE /api/employees/{id} (deleteById) to remove an employee.
7. Run the application, verify the employee table is auto-created in the database, insert a record, and query it back.

#### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                                            | Expected Response                                       |
|--------|--------------------|------------------------------------------------------------------|---------------------------------------------------------|
| POST   | /api/employees     | {<br>"empName":"Anita","department":"IT","salary":45000<br>}     | 201 Created — saved<br>Employee with generated<br>empId |
| GET    | /api/employees     | None                                                             | 200 OK — array of<br>Employees                          |
| GET    | /api/employees/1   | None                                                             | 200 OK — single<br>Employee                             |
| PUT    | /api/employees/1   | { "empName":"Anita<br>Sharma","department":"IT","salary":48000 } | 200 OK — updated<br>Employee                            |
| DELETE | /api/employees/1   | None                                                             | 204 No Content —<br>Employee removed                    |

## Practical No. 2: Spring Data Repository Hierarchy – CrudRepository vs PagingAndSortingRepository vs JpaRepository

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

### Objective:

To compare the three levels of the Spring Data repository hierarchy covered in this chapter — CrudRepository (basic CRUD), PagingAndSortingRepository (adds paging/sorting), and JpaRepository (adds JPA-specific batch operations) — by rewriting the same repository at each level and observing the additional methods available.

### Class Name(s) with Attributes:

#### **Product** (@Entity)

- productId : int (@Id @GeneratedValue)
- productName : String
- category : String
- price : double
- quantity : int

**ProductRepository** (progressively extends CrudRepository → PagingAndSortingRepository → JpaRepository)

#### **ProductRestController** (@RestController)

- productRepository : ProductRepository

### Step-by-Step Task Instructions:

1. Create the Product entity with the listed attributes.
2. First declare ProductRepository extending CrudRepository<Product, Integer> and expose basic save/findAll endpoints.
3. Change the interface to extend PagingAndSortingRepository<Product, Integer> and add a GET /api/products/sorted endpoint using findAll(Sort.by("price")).
4. Finally change it to extend JpaRepository<Product, Integer> and demonstrate a JPA-specific method such as saveAll() (batch insert) and flush().
5. Add a plain GET /api/products (findAll), PUT /api/products/{id} (fetch, modify, save) and DELETE /api/products/{id} (deleteById) so every level of the hierarchy is exercised through full CRUD.
6. Note in comments which methods became newly available at each stage of the hierarchy.
7. Test each version's endpoints in Postman before moving to the next stage.

### Postman API Testing:

| Method | Endpoint (Postman)                | Request Body / Params                                                                                                                                          | Expected Response                                              |
|--------|-----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| POST   | /api/products/batch               | [ {"productName":"Mouse","category":"Electronics","price":500,"quantity":20}, {"productName":"Keyboard","category":"Electronics","price":1200,"quantity":15} ] | 201 Created — saveAll() batch result (JpaRepository)           |
| GET    | /api/products                     | None                                                                                                                                                           | 200 OK — all products (CrudRepository.findAll)                 |
| GET    | /api/products/sorted?sortBy=price | None                                                                                                                                                           | 200 OK — products sorted by price (PagingAndSortingRepository) |
| PUT    | /api/products/1                   | { "productName":"Wireless Mouse","price":650 }                                                                                                                 | 200 OK — updated Product                                       |

| Method | Endpoint (Postman) | Request Body / Params | Expected Response                   |
|--------|--------------------|-----------------------|-------------------------------------|
| DELETE | /api/products/1    | None                  | 204 No Content —<br>Product removed |

## Practical No. 3: Custom Query Methods – findBy, countBy and @Query (JPQL)

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

### Objective:

To derive query methods automatically from method names (query derivation) and to write custom JPQL queries using @Query for requirements that cannot be expressed by naming conventions alone, as covered in the Spring Data JPA query-methods section of this chapter.

### Class Name(s) with Attributes:

**Employee** (reuse from Practical 1)

- empId, empName, department, salary

**EmployeeRepository** (additional derived & custom methods)

- List<Employee> findByDepartment(String department)
- long countByDepartment(String department)
- List<Employee> findBySalaryGreaterThan(double salary)
- @Query("SELECT e FROM Employee e WHERE e.salary BETWEEN :min AND :max")  
findBySalaryRange(min,max)

### Step-by-Step Task Instructions:

1. Extend EmployeeRepository with derived query methods findByDepartment() and countByDepartment().
2. Add findBySalaryGreaterThan(double salary) as another derived query method.
3. Write a custom JPQL query using @Query with named parameters to fetch employees within a salary range.
4. Expose each repository method through a corresponding endpoint in EmployeeRestController, accepting path variables / request parameters.
5. Add POST /api/employees to seed sample employee records across multiple departments and salary bands, PUT /api/employees/{id} to revise an employee's salary (so the salary-range query result changes), and DELETE /api/employees/{id} to remove one before re-running countByDepartment().
6. Test each query endpoint from Postman and verify correctness against the seeded data.

### Postman API Testing:

| Method | Endpoint (Postman)                              | Request Body / Params                                         | Expected Response                                               |
|--------|-------------------------------------------------|---------------------------------------------------------------|-----------------------------------------------------------------|
| POST   | /api/employees                                  | {<br>"empName":"Farhan","department":"IT","salary":52000<br>} | 201 Created — seed data for the query tests                     |
| GET    | /api/employees/department/IT                    | None                                                          | 200 OK — employees in IT department                             |
| GET    | /api/employees/count/IT                         | None                                                          | 200 OK — numeric count                                          |
| GET    | /api/employees/salary-above?salary=40000        | None                                                          | 200 OK — employees earning above 40000                          |
| GET    | /api/employees/salary-range?min=30000&max=60000 | None                                                          | 200 OK — employees within range                                 |
| PUT    | /api/employees/{id}                             | { "salary":65000 }                                            | 200 OK — updated salary now falls outside the 30000–60000 range |
| DELETE | /api/employees/{id}                             | None                                                          | 204 No Content — countByDepartment("IT") decreases by 1         |

## Practical No. 4: One-to-One Mapping – Three Worked Examples

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

### Objective:

To implement the One-to-One relationship using `@OneToOne`, `@JoinColumn` (owning side) and `mappedBy` (inverse side), with cascading so that saving a parent entity also persists its associated entity. Three independent one-to-one example pairs are implemented to reinforce the pattern across different domains; full CRUD is built and tested for the primary example (User–UserProfile).

### Class Name(s) with Attributes:

**User** (@Entity — Example 1, owning side)

- `userId : int (@Id @GeneratedValue)`
- `username : String`
- `email : String`
- `userProfile : UserProfile (@OneToOne, cascade=ALL, @JoinColumn(name="profile_id"))`

**UserProfile** (@Entity — Example 1, inverse side)

- `profileId : int (@Id @GeneratedValue)`
- `address : String`
- `phone : String`
- `dob : LocalDate`
- `user : User (@OneToOne(mappedBy="userProfile"))`

**Employee** (@Entity — Example 2, owning side)

- `empId : int (@Id @GeneratedValue)`
- `empName : String`
- `department : String`
- `passport : Passport (@OneToOne, cascade=ALL, @JoinColumn(name="passport_id"))`

**Passport** (@Entity — Example 2, inverse side)

- `passportId : int (@Id @GeneratedValue)`
- `passportNumber : String`
- `issuingCountry : String`
- `expiryDate : LocalDate`
- `employee : Employee (@OneToOne(mappedBy="passport"))`

**Country** (@Entity — Example 3, owning side)

- `countryId : int (@Id @GeneratedValue)`
- `countryName : String`
- `capital : Capital (@OneToOne, cascade=ALL, @JoinColumn(name="capital_id"))`

**Capital** (@Entity — Example 3, inverse side)

- `capitalId : int (@Id @GeneratedValue)`
- `cityName : String`
- `population : long`
- `country : Country (@OneToOne(mappedBy="capital"))`

### Step-by-Step Task Instructions:

1. Example 1 (User–UserProfile): Create the UserProfile entity, then the User entity holding a `@OneToOne` reference to UserProfile with `cascade = CascadeType.ALL` and `@JoinColumn(name="profile_id")`; add the inverse `mappedBy = "userProfile"` field on UserProfile.
2. Create UserRepository extending `JpaRepository<User, Integer>` and UserRestController exposing full CRUD: POST /api/users (nested profile JSON saved via cascade), GET /api/users and GET /api/users/{id}, PUT /api/users/{id} (update user + nested profile), and DELETE /api/users/{id} (cascades and removes the profile too).

3. Example 2 (Employee–Passport): Repeat the identical pattern — Employee is the owning side with a @OneToOne @JoinColumn(name="passport\_id") reference to Passport; Passport carries the inverse mappedBy = "passport" field. Create EmployeeRepository (or reuse) and expose the same style of CRUD endpoints under /api/employees-passport.
4. Example 3 (Country–Capital): Repeat the pattern once more — Country owns the @OneToOne @JoinColumn(name="capital\_id") reference to Capital; Capital carries mappedBy = "capital". Expose analogous CRUD endpoints under /api/countries.
5. For all three examples, confirm that saving the parent (User / Employee / Country) automatically persists the associated child (UserProfile / Passport / Capital) via cascading, and that deleting the parent removes the child as well.
6. Run full CRUD Postman tests (POST, GET, PUT, DELETE) against Example 1 in detail, then repeat at least the POST and GET steps for Examples 2 and 3 to confirm the pattern generalizes.

#### Postman API Testing:

| Method | Endpoint (Postman)      | Request Body / Params                                                                                                                   | Expected Response                                                  |
|--------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| POST   | /api/users              | { "username":"amit01","email":"amit@svgu.edu","userProfile": { "address":"Nanded","phone":"9999999999","dob":"2001-05-10" } }           | 201 Created — User with nested UserProfile including generated ids |
| GET    | /api/users/1            | None                                                                                                                                    | 200 OK — User object with embedded UserProfile                     |
| PUT    | /api/users/1            | { "email":"amit.new@svgu.edu","userProfile": { "address":"Pune","phone":"8888888888","dob":"2001-05-10" } }                             | 200 OK — updated User and nested UserProfile                       |
| DELETE | /api/users/1            | None                                                                                                                                    | 204 No Content — User and its cascaded UserProfile removed         |
| POST   | /api/employees-passport | { "empName":"Rakesh","department":"HR","passport": { "passportNumber":"N1234567","issuingCountry":"India","expiryDate":"2030-01-01" } } | 201 Created — Employee with nested Passport (Example 2)            |
| POST   | /api/countries          | { "countryName":"India","capital": { "cityName":"New Delhi","population":32941000 } }                                                   | 201 Created — Country with nested Capital (Example 3)              |

## Practical No. 5: One-to-Many / Many-to-One Mapping – Department and Employee

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

### Objective:

To implement a bidirectional One-to-Many (Department → Employees) and Many-to-One (Employee → Department) relationship using `mappedBy` on the parent side and `@JoinColumn` on the child side, and to persist a parent with a list of children in one request.

### Class Name(s) with Attributes:

**Department** (@Entity — parent, One side)

- deptId : int (@Id @GeneratedValue)
- deptName : String
- employees : List<Employee> (@OneToMany(mappedBy="department", cascade=ALL))

**Employee** (@Entity — child, Many side)

- empId : int (@Id @GeneratedValue)
- empName : String
- salary : double
- department : Department (@ManyToOne @JoinColumn(name="dept\_id"))

### Step-by-Step Task Instructions:

1. Modify (or recreate) the Employee entity to add a @ManyToOne field department with @JoinColumn(name = "dept\_id").
2. Create the Department entity with a @OneToMany(mappedBy = "department", cascade = CascadeType.ALL) list of employees.
3. Create DepartmentRepository extending JpaRepository<Department, Integer>.
4. Create DepartmentRestController exposing POST /api/departments that accepts a department name plus a list of employee objects, and saves the whole graph via cascading (remember to set employee.setDepartment(department) on each child before saving).
5. Add GET /api/departments/{id} to fetch a department with its employees, and GET /api/departments/{id}/employees to fetch only the employee list.
6. Add PUT /api/departments/{deptId}/employees/{empId} to reassign or update an existing employee's details within a department (Many-to-One navigation from Employee back to Department), and PUT /api/departments/{id} to rename a department.
7. Add DELETE /api/departments/{deptId}/employees/{empId} to remove a single employee from a department (Many-to-One side), and DELETE /api/departments/{id} to remove a department along with all its employees via cascade (One-to-Many side).
8. Test the parent-with-children creation, both read endpoints, both update endpoints, and both delete endpoints in Postman.

### Postman API Testing:

| Method | Endpoint (Postman)           | Request Body / Params                                                                                                             | Expected Response                                                                         |
|--------|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| POST   | /api/departments             | { "deptName":"Computer Science",<br>"employees":[<br>{"empName":"Riya","salary":42000},<br>{"empName":"Karan","salary":39000} ] } | 201 Created — Department with generated deptId and employees carrying dept_id foreign key |
| GET    | /api/departments/1           | None                                                                                                                              | 200 OK — Department with nested employees list                                            |
| GET    | /api/departments/1/employees | None                                                                                                                              | 200 OK — list of Employees belonging to department 1 (Many-to-One → One side)             |

| Method | Endpoint (Postman)             | Request Body / Params                      | Expected Response                                                                         |
|--------|--------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------|
| PUT    | /api/departments/1/employees/2 | { "empName":"Karan Patil","salary":41000 } | 200 OK — updated Employee within Department 1 (Many-to-One update)                        |
| DELETE | /api/departments/1/employees/2 | None                                       | 204 No Content — Employee removed, Department retains remaining employees                 |
| DELETE | /api/departments/1             | None                                       | 204 No Content — Department and all remaining employees removed via cascade (One-to-Many) |

## Practical No. 6: Many-to-Many Mapping – Student and Course

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

### Objective:

To implement a Many-to-Many relationship between Student and Course using `@ManyToMany` and a join table via `@JoinTable`, and to avoid infinite JSON recursion using `@JsonManagedReference` / `@JsonBackReference`.

### Class Name(s) with Attributes:

#### **Student** (@Entity)

- studentId : int (@Id @GeneratedValue)
- studentName : String
- courses : Set<Course> (@ManyToMany, @JoinTable(name="student\_course"))

#### **Course** (@Entity)

- courseId : int (@Id @GeneratedValue)
- courseName : String
- students : Set<Student> (@ManyToMany(mappedBy="courses"))

### Step-by-Step Task Instructions:

1. Create the Course entity with the inverse mapping students, using mappedBy = "courses".
2. Create the Student entity as the owning side with `@ManyToMany` and `@JoinTable(name = "student_course", joinColumns = @JoinColumn(name="student_id"), inverseJoinColumns = @JoinColumn(name="course_id"))`.
3. Annotate the owning collection with `@JsonManagedReference` and the inverse collection with `@JsonBackReference` to prevent infinite serialization loops.
4. Create StudentRepository and CourseRepository extending JpaRepository.
5. Create StudentRestController with POST /api/students accepting a student along with a set of existing course ids/objects, and a separate endpoint POST /api/students/{id}/enroll/{courseId} to enrol an existing student into an additional course.
6. Add GET /api/students/{id} to view a student with their enrolled courses, PUT /api/courses/{id} to update a course's details, and PUT /api/students/{id} to replace a student's entire course set.
7. Add DELETE /api/students/{studentId}/unenroll/{courseId} to remove a single student-course link from the join table (without deleting either entity), and DELETE /api/courses/{id} to delete a course entirely (removing its join-table rows first).
8. Test creation with multiple courses and the enrol-into-additional-course endpoint, then confirm from the Course side which students are enrolled, and finally test the update and unenroll/delete endpoints.

### Postman API Testing:

| Method | Endpoint (Postman)         | Request Body / Params                                        | Expected Response                                                            |
|--------|----------------------------|--------------------------------------------------------------|------------------------------------------------------------------------------|
| POST   | /api/courses               | { "courseName": "Advance Java" }                             | 201 Created                                                                  |
| POST   | /api/students              | { "studentName": "Priya", "courses": [ { "courseId": 1 } ] } | 201 Created — Student linked to Course via join table                        |
| POST   | /api/students/1/enroll/2   | None                                                         | 200 OK — Student now linked to an additional course                          |
| GET    | /api/students/1            | None                                                         | 200 OK — Student with the set of enrolled Courses                            |
| GET    | /api/courses/1/students    | None                                                         | 200 OK — all students enrolled in course 1                                   |
| PUT    | /api/courses/1             | { "courseName": "Advance Java (Spring Boot)" }               | 200 OK — updated Course, join-table links unaffected                         |
| DELETE | /api/students/1/unenroll/2 | None                                                         | 204 No Content — join-table row removed; Student and Course both still exist |

| Method | Endpoint (Postman) | Request Body / Params | Expected Response                                                 |
|--------|--------------------|-----------------------|-------------------------------------------------------------------|
| DELETE | /api/courses/1     | None                  | 204 No Content — Course removed along with its join-table entries |

## Practical No. 7: Transaction Management with @Transactional – Bank Fund Transfer

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

### Objective:

To understand declarative transaction management in Spring Data JPA, and to ensure atomicity across multiple repository operations (debit + credit) using @Transactional, including automatic rollback when a runtime exception occurs.

### Class Name(s) with Attributes:

#### *Account* (@Entity)

- accountId : int (@Id @GeneratedValue)
- accountHolder : String
- balance : double

#### *InsufficientBalanceException* (custom RuntimeException)

#### *AccountService* / *AccountServiceImpl* (transferFunds is @Transactional)

- accountRepository : AccountRepository

### Step-by-Step Task Instructions:

1. Create the Account entity and AccountRepository extending JpaRepository<Account, Integer>.
2. Create a custom unchecked exception InsufficientBalanceException.
3. In AccountServiceImpl, write method transferFunds(fromId, toId, amount) annotated with @Transactional that: fetches the source account, checks balance, debits it, fetches the destination account, credits it, and saves both.
4. Deliberately throw InsufficientBalanceException after the debit step (or simulate by transferring more than the available balance) to prove that @Transactional rolls back the debit too.
5. Expose POST /api/accounts/transfer in a REST controller accepting fromId, toId, and amount.
6. Add GET /api/accounts/{id} to view a single account's balance, PUT /api/accounts/{id} to update non-transactional account details such as accountHolder name, and DELETE /api/accounts/{id} to close an account (only permitted when its balance is zero — throw a custom exception otherwise).
7. Test a successful transfer, then test a failing transfer, and re-check both balances via GET to confirm rollback left them unchanged.

### Postman API Testing:

| Method | Endpoint (Postman)     | Request Body / Params                     | Expected Response                                                               |
|--------|------------------------|-------------------------------------------|---------------------------------------------------------------------------------|
| POST   | /api/accounts/transfer | { "fromId":1, "toId":2, "amount":5000 }   | 200 OK — both balances updated                                                  |
| POST   | /api/accounts/transfer | { "fromId":1, "toId":2, "amount":999999 } | 400/409 with error message — balances unchanged, verified via GET /api/accounts |
| GET    | /api/accounts/1        | None                                      | 200 OK — current balance for account 1                                          |
| PUT    | /api/accounts/1        | { "accountHolder":"Rohan Mehta" }         | 200 OK — accountHolder updated, balance untouched                               |
| DELETE | /api/accounts/2        | None                                      | 204 No Content if balance is zero, otherwise 400 Bad Request                    |

## Practical No. 8: Pagination and Sorting with Spring Data JPA

**Textbook Reference:** *Chapter 5 — Working with Spring Data JPA*

### Objective:

To implement server-side pagination and sorting for a large result set using Spring Data's Pageable/PageRequest abstraction (PagingAndSortingRepository capability), exposed through query parameters on a REST endpoint.

### Class Name(s) with Attributes:

**Product** (reuse from Practical 2)

- productId, productName, category, price, quantity

**ProductRestController** (extended)

- Method: getProducts(page,size,sortBy) : Page<Product>

### Step-by-Step Task Instructions:

1. Insert at least 15–20 sample Product records for meaningful pagination.
2. Ensure ProductRepository extends JpaRepository (which supports findAll(Pageable)).
3. In ProductRestController, add GET /api/products/page accepting @RequestParam page, size, and sortBy (with defaults).
4. Inside the method, build Pageable pageable = PageRequest.of(page, size, Sort.by(sortBy)) and call productRepository.findAll(pageable).
5. Return the resulting Page<Product> directly (Spring serializes content, totalPages, totalElements, etc. automatically).
6. Add POST /api/products (create one more record and observe it appear on the correct page), PUT /api/products/{id} (update a record's price/category and re-sort), and DELETE /api/products/{id} (remove a record and confirm totalElements decreases).
7. Test with different page/size/sortBy combinations in Postman and verify totalPages and totalElements are correct after each create/update/delete.

### Postman API Testing:

| Method | Endpoint (Postman)                                  | Request Body / Params                                                                  | Expected Response                                                     |
|--------|-----------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| GET    | /api/products/page?page=0&size=5&sortBy=price       | None                                                                                   | 200 OK — JSON with content[], totalPages, totalElements, number, size |
| GET    | /api/products/page?page=1&size=5&sortBy=productName | None                                                                                   | 200 OK — second page sorted alphabetically by productName             |
| POST   | /api/products                                       | {<br>"productName":"Monitor","category":"Electronics","price":9500,"quantity":8 }<br>} | 201 Created — totalElements increases by 1 on next page fetch         |
| PUT    | /api/products/{id}                                  | { "price":8999 }                                                                       | 200 OK — updated price reflected in re-sorted page                    |
| DELETE | /api/products/{id}                                  | None                                                                                   | 204 No Content — totalElements decreases by 1                         |



## Chapter 6 — Building RESTful Microservices

### Practical No. 9: Building a RESTful Microservice using @RestController and ResponseEntity

**Textbook Reference:** *Chapter 6 — Building RESTful Microservices*

#### Objective:

To design and build a RESTful microservice endpoint using @RestController, mapping HTTP verbs to CRUD operations, and using ResponseEntity to control the HTTP status code and headers returned to the client — the core REST-building technique introduced in this chapter.

#### Class Name(s) with Attributes:

##### **Book** (POJO)

- id : int
- title : String
- author : String
- price : double

##### **BookRestController** (@RestController @RequestMapping("/api/books"))

- Methods return ResponseEntity<Book> / ResponseEntity<List<Book>>

#### Step-by-Step Task Instructions:

1. Create the Book POJO with the listed attributes and an in-memory List<Book> store inside the controller/service for this practical.
2. Implement POST /api/books returning ResponseEntity.status(HttpStatus.CREATED).body(savedBook).
3. Implement GET /api/books returning ResponseEntity.ok(bookList).
4. Implement GET /api/books/{id} returning ResponseEntity.ok(book) when found, or ResponseEntity.notFound().build() when absent.
5. Implement PUT /api/books/{id} and DELETE /api/books/{id}, each returning an appropriate ResponseEntity and HTTP status.
6. Test every endpoint in Postman and specifically observe the HTTP status code returned for each case (201, 200, 404, 204).

#### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                                             | Expected Response                           |
|--------|--------------------|-------------------------------------------------------------------|---------------------------------------------|
| POST   | /api/books         | { "title":"Spring in Action","author":"Craig Walls","price":650 } | 201 Created — Location/body with saved Book |
| GET    | /api/books/1       | None                                                              | 200 OK, or 404 Not Found if absent          |
| PUT    | /api/books/1       | { "title":"Spring in Action 6E","price":700 }                     | 200 OK — updated Book                       |
| DELETE | /api/books/1       | None                                                              | 204 No Content                              |

## Practical No. 10: Layered Microservice Architecture – RestController + Service + JPA Repository

**Textbook Reference:** *Chapter 6 — Building RESTful Microservices*

### Objective:

To structure a RESTful microservice into the standard Controller–Service–Repository layers recommended in this chapter, keeping business logic out of the controller and persistence access out of the service.

### Class Name(s) with Attributes:

**Product** (@Entity, reused)

- productId, productName, category, price, quantity

**ProductService** (interface)

- CRUD method signatures

**ProductServiceImpl** (@Service)

- productRepository : ProductRepository

**ProductRestController** (@RestController)

- productService : ProductService

### Step-by-Step Task Instructions:

1. Create ProductService (interface) declaring create, getAll, getById, update, delete methods.
2. Create ProductServiceImpl annotated @Service, injecting ProductRepository and implementing the interface.
3. Create ProductRestController injecting ProductService only (never the repository directly), exposing the full CRUD REST API.
4. For update, fetch the existing entity by id in the service layer, apply changes, and call save() again.
5. Confirm via code review that the controller has no direct repository reference (proper layering).
6. Test the complete CRUD lifecycle in Postman, saving requests into a Postman Collection for reuse.

### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                                                                 | Expected Response               |
|--------|--------------------|---------------------------------------------------------------------------------------|---------------------------------|
| POST   | /api/products      | {<br>"productName":"Laptop","category":"Electronics","price":55000,"quantity":10<br>} | 201 Created                     |
| GET    | /api/products      | None                                                                                  | 200 OK —<br>list of<br>products |
| PUT    | /api/products/{id} | { "productName":"Laptop Pro","price":60000 }                                          | 200 OK —<br>updated<br>product  |
| DELETE | /api/products/{id} | None                                                                                  | 204 No<br>Content               |

## Practical No. 11: Handling Request Data – @PathVariable, @RequestParam and @RequestBody

**Textbook Reference:** *Chapter 6 — Building RESTful Microservices*

### Objective:

To correctly use the three main ways a Spring REST controller receives client input — @PathVariable for identifying a resource, @RequestParam for optional query filters, and @RequestBody for JSON payloads — as covered in the request-handling section of this chapter.

### Class Name(s) with Attributes:

#### **Customer** (@Entity)

- customerId : int (@Id @GeneratedValue)
- name : String
- city : String
- age : int

#### **CustomerRestController** (@RestController)

- Method: getById(@PathVariable id)
- Method: search(@RequestParam city, @RequestParam(required=false) minAge)
- Method: create(@RequestBody CustomerDTO)

### Step-by-Step Task Instructions:

1. Create the Customer entity and CustomerRepository extending JpaRepository.
2. Implement GET /api/customers/{id} using @PathVariable to fetch a single customer.
3. Implement GET /api/customers/search?city=..&minAge=.. using @RequestParam (with minAge optional and defaulting to 0).
4. Implement POST /api/customers accepting the full Customer object via @RequestBody.
5. Implement PUT /api/customers/{id} (combining @PathVariable for the id with @RequestBody for the updated fields) and DELETE /api/customers/{id} (using @PathVariable alone) to complete the CRUD set.
6. Add logging (System.out / SLF4J) inside each handler to print which parameter source was used, for demonstration.
7. Test all three parameter-passing styles from Postman using the URL bar, query params tab, and body tab respectively.

### Postman API Testing:

| Method | Endpoint (Postman)                        | Request Body / Params                                  | Expected Response                                        |
|--------|-------------------------------------------|--------------------------------------------------------|----------------------------------------------------------|
| GET    | /api/customers/3                          | None (@PathVariable)                                   | 200 OK — Customer with id 3                              |
| GET    | /api/customers/search?city=Pune&minAge=25 | None (@RequestParam)                                   | 200 OK — filtered list of customers                      |
| POST   | /api/customers                            | {<br>"name": "Meera", "city": "Nanded", "age": 22<br>} | 201 Created (@RequestBody)                               |
| PUT    | /api/customers/3                          | { "name": "Meera Joshi", "city": "Pune", "age": 23 }   | 200 OK — updated Customer (@PathVariable + @RequestBody) |
| DELETE | /api/customers/3                          | None (@PathVariable)                                   | 204 No Content                                           |

## Practical No. 12: DTO Pattern – Separating the Entity from the API Contract

**Textbook Reference:** *Chapter 6 — Building RESTful Microservices*

### Objective:

To decouple the persistent JPA entity from the data actually exposed by the REST API using the Data Transfer Object (DTO) pattern discussed in this chapter, hiding sensitive/internal fields and mapping between Entity and DTO.

### Class Name(s) with Attributes:

**Employee** (@Entity — add sensitive field)

- empId, empName, department, salary
- password : String (sensitive — never exposed)

**EmployeeDTO** (POJO)

- empId : int
- empName : String
- department : String

**EmployeeMapper** (utility class)

- static EmployeeDTO toDto(Employee e)
- static Employee toEntity(EmployeeDTO d)

### Step-by-Step Task Instructions:

1. Add a password field to the Employee entity to represent sensitive data that should never leave the server.
2. Create the EmployeeDTO class containing only the fields safe to expose.
3. Create EmployeeMapper with static conversion methods toDto() and toEntity().
4. Modify EmployeeRestController's GET endpoints to convert Employee → EmployeeDTO using the mapper before returning the response.
5. Modify the POST endpoint to accept an EmployeeDTO, convert it to an entity, and save.
6. Add PUT /api/employees/{id} that accepts an EmployeeDTO, maps it onto the existing entity (preserving the untouched password field), and saves; add DELETE /api/employees/{id} to remove the record.
7. Test GET /api/employees and confirm the password field is absent from the JSON response.

### Postman API Testing:

| Method | Endpoint (Postman)  | Request Body / Params                                | Expected Response                                                         |
|--------|---------------------|------------------------------------------------------|---------------------------------------------------------------------------|
| POST   | /api/employees      | {<br>"empName":"Ibrahim","department":"Finance"<br>} | 201 Created —<br>EmployeeDTO returned (no password field)                 |
| GET    | /api/employees      | None                                                 | 200 OK — JSON contains empId, empName, department only; no password field |
| PUT    | /api/employees/{id} | { "empName":"Ibrahim Khan","department":"Finance" }  | 200 OK — updated EmployeeDTO; password remains untouched internally       |
| DELETE | /api/employees/{id} | None                                                 | 204 No Content                                                            |

## Practical No. 13: Request Validation using Bean Validation (@Valid) with RestController

**Textbook Reference:** *Chapter 6 — Building RESTful Microservices*

### Objective:

To validate incoming JSON payloads declaratively using Jakarta Bean Validation annotations (@NotBlank, @Email, @Min) combined with @Valid on the controller method, as covered in this chapter's discussion of building robust REST endpoints.

### Class Name(s) with Attributes:

**Customer** (@Entity / DTO, reused)

- customerId : int (@Id @GeneratedValue)
- name : String (@NotBlank)
- email : String (@Email)
- age : int (@Min(18))

### Step-by-Step Task Instructions:

1. Add spring-boot-starter-validation dependency.
2. Annotate the Customer class fields with @NotBlank, @Email and @Min(18) as listed.
3. In CustomerRestController, annotate the @RequestBody parameter of the create method with @Valid.
4. Run the application and POST invalid data (blank name, bad email format, age below 18) — observe Spring's default 400 Bad Request response.
5. Note that field-level error detail will be structured properly once GlobalExceptionHandler (next practical) is added.
6. Add GET /api/customers/{id} to fetch a saved customer, and PUT /api/customers/{id} annotated with @Valid on its @RequestBody so updates are validated the same way as creation; add DELETE /api/customers/{id} to remove a customer.
7. Test again with fully valid data and confirm successful creation (201), then test an invalid PUT update and a valid DELETE.

### Postman API Testing:

| Method | Endpoint (Postman)  | Request Body / Params                                | Expected Response                                              |
|--------|---------------------|------------------------------------------------------|----------------------------------------------------------------|
| POST   | /api/customers      | { "name":"","email":"bad-email", "age":15 }          | 400 Bad Request — validation failure                           |
| POST   | /api/customers      | { "name":"Sneha","email":"sneha@svgu.edu","age":21 } | 201 Created                                                    |
| GET    | /api/customers/{id} | None                                                 | 200 OK — saved Customer                                        |
| PUT    | /api/customers/{id} | { "name":"Sneha","email":"not-an-email","age":21 }   | 400 Bad Request — @Valid rejects the malformed email on update |
| DELETE | /api/customers/{id} | None                                                 | 204 No Content                                                 |

## Practical No. 14: Global Exception Handling using @ControllerAdvice and @ExceptionHandler

**Textbook Reference:** *Chapter 6 — Building RESTful Microservices*

### Objective:

To centralize exception handling across all REST controllers using @ControllerAdvice, returning a consistent, structured error response (timestamp, message, status, path) instead of default Spring stack-trace pages — the concluding topic of this chapter.

### Class Name(s) with Attributes:

**ResourceNotFoundException** (custom RuntimeException)

**ErrorResponse** (POJO)

- timestamp : LocalDateTime
- message : String
- status : int
- path : String

**GlobalExceptionHandler** (@ControllerAdvice)

- Method: handleNotFound(ResourceNotFoundException)
- Method: handleValidation(MethodArgumentNotValidException)
- Method: handleGenericException(Exception)

### Step-by-Step Task Instructions:

1. Create ResourceNotFoundException extending RuntimeException.
2. Create the ErrorResponse POJO with the listed fields.
3. In the service layer (e.g., EmployeeService or ProductService), throw ResourceNotFoundException when findById() returns empty instead of returning null.
4. Create GlobalExceptionHandler annotated @ControllerAdvice with an @ExceptionHandler(ResourceNotFoundException.class) method returning ResponseEntity<ErrorResponse> with HTTP 404.
5. Add an @ExceptionHandler(MethodArgumentNotValidException.class) method that collects field-wise validation errors from Practical 13 into a structured response with HTTP 400.
6. Add a catch-all @ExceptionHandler(Exception.class) method returning HTTP 500 for any unhandled exception, and re-test the invalid Customer payload and a non-existent Employee id.
7. Make sure PUT /api/employees/{id} and DELETE /api/employees/{id} also route through ResourceNotFoundException (not a raw NullPointerException) when the id does not exist, and re-test all four HTTP verbs against a non-existent id.

### Postman API Testing:

| Method | Endpoint (Postman)  | Request Body / Params                           | Expected Response                                                  |
|--------|---------------------|-------------------------------------------------|--------------------------------------------------------------------|
| GET    | /api/employees/9999 | None                                            | 404 Not Found — JSON { timestamp, message, status, path }          |
| POST   | /api/customers      | { "name": "", "email": "bad-email", "age": 15 } | 400 Bad Request — structured field-error JSON via ControllerAdvice |
| PUT    | /api/employees/9999 | { "empName": "Ghost" }                          | 404 Not Found — structured error, not a stack trace                |
| DELETE | /api/employees/9999 | None                                            | 404 Not Found — structured error via GlobalExceptionHandler        |

## Practical No. 15: Securing REST API with HTTP Basic Authentication (Spring Security)

**Textbook Reference:** *Chapter 7 — Securing a Web Application*

### Objective:

To add Spring Security to a Spring Boot REST application, configure in-memory users with encoded passwords, and secure selected endpoints so that only authenticated requests can access them — the introductory security mechanism covered in this chapter.

### Class Name(s) with Attributes:

**SecurityConfig** (@Configuration)

- Bean: SecurityFilterChain filterChain()
- Bean: PasswordEncoder passwordEncoder() — BCryptPasswordEncoder
- Bean: InMemoryUserDetailsManager userDetailsService()

### Step-by-Step Task Instructions:

1. Add spring-boot-starter-security dependency to pom.xml.
2. Create SecurityConfig with a PasswordEncoder bean using BCryptPasswordEncoder.
3. Define a UserDetailsService bean with at least one in-memory user (username 'user', BCrypt-encoded password, role USER).
4. Define a SecurityFilterChain bean that permits access to /api/public/\*\* without authentication and requires authentication for /api/secure/\*\*, enabling httpBasic().
5. Create two endpoints: GET /api/public/data (unsecured) and GET /api/secure/data (secured).
6. Add POST /api/secure/data, PUT /api/secure/data/{id} and DELETE /api/secure/data/{id} to a simple secured resource (e.g., an in-memory Note list) so that every CRUD verb — not just GET — is protected by the same SecurityFilterChain rule.
7. Test /api/secure/data without credentials (expect 401) and then with correct Basic Auth credentials configured in Postman's Authorization tab (expect 200), then repeat for POST/PUT/DELETE.

### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                              | Expected Response                           |
|--------|--------------------|----------------------------------------------------|---------------------------------------------|
| GET    | /api/secure/data   | None (no auth)                                     | 401 Unauthorized                            |
| GET    | /api/secure/data   | Authorization tab → Basic Auth → username/password | 200 OK — secured data returned              |
| POST   | /api/secure/data   | Basic Auth + { "text":"Meeting at 10am" }          | 201 Created — only with valid Basic Auth    |
| PUT    | /api/secure/data/1 | Basic Auth + { "text":"Meeting at 11am" }          | 200 OK — only with valid Basic Auth         |
| DELETE | /api/secure/data/1 | Basic Auth                                         | 204 No Content — only with valid Basic Auth |
| GET    | /api/public/data   | None                                               | 200 OK — accessible without login           |

## Practical No. 16: User Registration and Login with BCrypt Password Encoding

**Textbook Reference:** *Chapter 7 — Securing a Web Application*

### Objective:

To implement a database-backed user registration and login mechanism, storing passwords securely using BCryptPasswordEncoder instead of plain text, and authenticating against the database via a custom UserDetailsService — extending the password-encoding concept from this chapter to a persistent user store.

### Class Name(s) with Attributes:

#### **AppUser** (@Entity)

- userId : int (@Id @GeneratedValue)
- username : String (unique)
- password : String (BCrypt-encoded)
- role : String

#### **AppUserRepository** (extends JpaRepository<AppUser, Integer>)

- Optional<AppUser> findByUsername(String username)

#### **CustomUserDetailsService** (implements UserDetailsService)

- appUserRepository : AppUserRepository

#### **AuthController** (@RestController)

- Method: register(@RequestBody AppUser) — encodes password before saving

### Step-by-Step Task Instructions:

1. Create the AppUser entity and AppUserRepository with a findByUsername() derived query method.
2. Create CustomUserDetailsService implementing UserDetailsService, loading a user by username and returning a Spring Security User object built from the stored (encoded) password and role.
3. Update SecurityConfig to use CustomUserDetailsService instead of the in-memory user store from Practical 15, with an AuthenticationProvider wired to the BCryptPasswordEncoder bean.
4. Create AuthController with POST /api/auth/register that encodes the incoming plain-text password using passwordEncoder.encode() before saving the AppUser.
5. Register a new user via Postman, then inspect the database to confirm the stored password is a BCrypt hash, not plain text.
6. Add PUT /api/auth/profile (secured, Basic Auth) to let the authenticated user update their own username/role, and DELETE /api/auth/deregister (secured) to remove the authenticated user's account.
7. Test HTTP Basic Auth login against a secured endpoint using the newly registered username and original (plain-text) password, then test the profile update and deregistration.

### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                                                   | Expected Response                                        |
|--------|--------------------|-------------------------------------------------------------------------|----------------------------------------------------------|
| POST   | /api/auth/register | {<br>"username": "neha01", "password": "MyP@ss123", "role": "USER"<br>} | 201 Created — user saved with BCrypt-hashed password     |
| GET    | /api/secure/data   | Authorization tab → Basic Auth → neha01 / MyP@ss123                     | 200 OK — authenticated against BCrypt-hashed DB password |
| PUT    | /api/auth/profile  | Basic Auth: neha01/MyP@ss123 + {<br>"role": "USER_PREMIUM" }            | 200 OK — role updated for the authenticated user         |

| Method | Endpoint (Postman)   | Request Body / Params        | Expected Response                                        |
|--------|----------------------|------------------------------|----------------------------------------------------------|
| DELETE | /api/auth/deregister | Basic Auth: neha01/MyP@ss123 | 204 No Content —<br>account removed<br>from the database |

## Practical No. 17: Role-Based Access Control (RBAC) with Spring Security

**Textbook Reference:** *Chapter 7 — Securing a Web Application*

### Objective:

To extend Spring Security configuration for role-based authorization, defining users with different roles (ADMIN, USER) and restricting specific endpoints using `@PreAuthorize` or role-based matchers, as covered in this chapter's authorization section.

### Class Name(s) with Attributes:

#### **SecurityConfig** (extended)

- Users carrying `ROLE_ADMIN` / `ROLE_USER`
- `@EnableMethodSecurity` enabled

#### **AdminController** (`@RestController`)

- Method: `getAdminData()` — `@PreAuthorize("hasRole('ADMIN')")`

### Step-by-Step Task Instructions:

1. Ensure at least one `AppUser` (or in-memory user) has role `ADMIN` and another has role `USER`.
2. Add `@EnableMethodSecurity` on the `SecurityConfig` class.
3. Create `AdminController` with `GET /api/admin/data` annotated `@PreAuthorize("hasRole('ADMIN')")`.
4. Create a second endpoint `GET /api/user/data` annotated `@PreAuthorize("hasAnyRole('ADMIN','USER')")` accessible to both roles.
5. Add `POST /api/admin/data`, `PUT /api/admin/data/{id}` and `DELETE /api/admin/data/{id}`, all annotated `@PreAuthorize("hasRole('ADMIN')")`, managing a simple in-memory admin-only resource.
6. Test `/api/admin/data` first with `USER` credentials (expect 403 Forbidden) and then with `ADMIN` credentials (expect 200), and repeat the same `USER`-vs-`ADMIN` comparison for the `POST`, `PUT` and `DELETE` endpoints.
7. Test `/api/user/data` with both sets of credentials and confirm both succeed.

### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                                    | Expected Response                         |
|--------|--------------------|----------------------------------------------------------|-------------------------------------------|
| GET    | /api/admin/data    | Basic Auth: user/pass ( <code>ROLE_USER</code> )         | 403 Forbidden                             |
| GET    | /api/admin/data    | Basic Auth: admin/pass ( <code>ROLE_ADMIN</code> )       | 200 OK                                    |
| POST   | /api/admin/data    | Basic Auth: admin/pass + { "label": "Policy Update" }    | 201 Created — ADMIN only, 403 for USER    |
| PUT    | /api/admin/data/1  | Basic Auth: admin/pass + { "label": "Policy Update v2" } | 200 OK — ADMIN only, 403 for USER         |
| DELETE | /api/admin/data/1  | Basic Auth: admin/pass                                   | 204 No Content — ADMIN only, 403 for USER |
| GET    | /api/user/data     | Basic Auth: user/pass                                    | 200 OK                                    |

## Practical No. 18: Securing REST API as an OAuth2 Resource Server with JWT

**Textbook Reference:** *Chapter 7 — Securing a Web Application*

### Objective:

To configure the Spring Boot application as an OAuth2 Resource Server that validates JSON Web Tokens (JWT) issued by an Authorization Server, protecting REST endpoints so that only requests carrying a valid Bearer token succeed — the token-based security mechanism covered in this chapter.

### Class Name(s) with Attributes:

**ResourceServerConfig** (@Configuration — OAuth2 resource server)

- Bean: SecurityFilterChain configured with oauth2ResourceServer().jwt()

**SecureDataController** (@RestController)

- Method: getSecureData() — requires valid JWT with configured scope

### Step-by-Step Task Instructions:

1. Add spring-boot-starter-oauth2-resource-server dependency to pom.xml.
2. Configure spring.security.oauth2.resourceserver.jwt.issuer-uri (or jwk-set-uri) in application.properties, pointing to an Authorization Server.
3. In ResourceServerConfig, configure the SecurityFilterChain to require authentication for /api/secure/\*\* using .oauth2ResourceServer(oauth2 -> oauth2.jwt()).
4. Obtain a sample JWT access token from the configured Authorization Server for testing.
5. Create GET /api/secure/data returning protected data, optionally checking a required scope/authority using @PreAuthorize("hasAuthority('SCOPE\_read')").
6. Add POST /api/secure/data, PUT /api/secure/data/{id} and DELETE /api/secure/data/{id}, each requiring the same valid JWT (and, optionally, a 'write' scope for the mutating verbs) via .oauth2ResourceServer(oauth2 -> oauth2.jwt()).
7. In Postman, call each endpoint without a token, then with an Authorization: Bearer <token> header, and finally with a deliberately invalid/expired token to observe all three outcomes for every verb.

### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                                                                   | Expected Response                      |
|--------|--------------------|-----------------------------------------------------------------------------------------|----------------------------------------|
| GET    | /api/secure/data   | No Authorization header                                                                 | 401 Unauthorized                       |
| GET    | /api/secure/data   | Header: Authorization: Bearer <valid JWT>                                               | 200 OK — protected resource data       |
| POST   | /api/secure/data   | Header: Authorization: Bearer <valid JWT with write scope> + { "text": "New note" }     | 201 Created                            |
| PUT    | /api/secure/data/1 | Header: Authorization: Bearer <valid JWT with write scope> + { "text": "Updated note" } | 200 OK                                 |
| DELETE | /api/secure/data/1 | Header: Authorization: Bearer <valid JWT with write scope>                              | 204 No Content                         |
| GET    | /api/secure/data   | Header: Authorization: Bearer <invalid/expired JWT>                                     | 401 Unauthorized — invalid_token error |

## Practical No. 19: OAuth2 Login with a Third-Party Provider (Google/GitHub) and Accessing a Secured REST API

**Textbook Reference:** *Chapter 7 — Securing a Web Application*

### Objective:

To implement the OAuth2 Authorization Code grant using Spring Security's OAuth2 Client support with a real provider (Google or GitHub), obtain an access token, and use that token to call a secured REST endpoint — completing this chapter's end-to-end OAuth2 flow using Postman's built-in OAuth 2.0 authorization support.

### Class Name(s) with Attributes:

**OAuth2ClientConfig** (*application.properties based client registration*)

- spring.security.oauth2.client.registration.google.client-id
- spring.security.oauth2.client.registration.google.client-secret
- spring.security.oauth2.client.registration.google.scope

**UserInfoController** (*@RestController*)

- Method: getUserProfile(@AuthenticationPrincipal OAuth2User principal) : Map<String,Object>

### Step-by-Step Task Instructions:

1. Register a new OAuth application on the Google Cloud Console (or GitHub Developer Settings) to obtain a Client ID and Client Secret; set the authorized redirect URI to `http://localhost:8080/login/oauth2/code/google`.
2. Add `spring-boot-starter-oauth2-client` dependency to `pom.xml`.
3. Configure `spring.security.oauth2.client.registration.google.client-id`, `client-secret` and `scope` (`openid`, `profile`, `email`) in `application.properties`.
4. Create `UserInfoController` with `GET /api/user/profile` that reads the authenticated `OAuth2User` principal and returns selected attributes (`name`, `email`) as JSON.
5. Run the application, browse to `http://localhost:8080/oauth2/authorization/google` to complete the login/consent flow, and confirm redirection back with an authenticated session.
6. Add `POST /api/user/notes`, `PUT /api/user/notes/{id}` and `DELETE /api/user/notes/{id}` as simple session-authenticated endpoints (any user who completed OAuth2 login may manage their own notes), to demonstrate that the OAuth2-authenticated principal can perform full CRUD, not just reads.
7. In Postman, open the Authorization tab, select Type = OAuth 2.0, and enter the Auth URL, Token URL, Client ID, Client Secret, and Redirect URI matching the provider's values; click 'Get New Access Token', then call `GET /api/user/profile` using the retrieved token.
8. Reuse the same retrieved token to call `POST`, `PUT` and `DELETE` on `/api/user/notes` and confirm each succeeds only while the token is valid.

### Postman API Testing:

| Method | Endpoint (Postman)                                   | Request Body / Params                                        | Expected Response                                      |
|--------|------------------------------------------------------|--------------------------------------------------------------|--------------------------------------------------------|
| —      | Authorization tab → OAuth 2.0 → Get New Access Token | Auth URL, Token URL, Client ID/Secret, Redirect URI, Scope   | Access token retrieved and stored by Postman           |
| GET    | /api/user/profile                                    | Authorization: Bearer <access token from Postman OAuth2>     | 200 OK — JSON with authenticated user's name and email |
| POST   | /api/user/notes                                      | Bearer <token> + { "text":"Follow up with client" }          | 201 Created — note saved for the authenticated user    |
| PUT    | /api/user/notes/1                                    | Bearer <token> + { "text":"Follow up with client tomorrow" } | 200 OK — note updated                                  |
| DELETE | /api/user/notes/1                                    | Bearer <token>                                               | 204 No Content — note removed                          |

## Practical No. 20: Cross-Origin Resource Sharing (CORS) Configuration for Secured REST APIs

**Textbook Reference:** *Chapter 7 — Securing a Web Application*

### Objective:

To configure CORS at the application level so that a secured REST API can be safely consumed by a browser-based frontend hosted on a different origin — a practical security configuration concern addressed alongside authentication/authorization in this chapter.

### Class Name(s) with Attributes:

**CorsConfig** (@Configuration implements WebMvcConfigurer)

- Method: addCorsMappings(CorsRegistry registry)

**SecurityConfig** (extended)

- .cors(Customizer.withDefaults()) enabled in the SecurityFilterChain

### Step-by-Step Task Instructions:

1. Create CorsConfig implementing WebMvcConfigurer and override addCorsMappings() to allow origin http://localhost:3000 (a sample frontend) on /api/\*\* with GET, POST, PUT, DELETE methods.
2. Update SecurityConfig's SecurityFilterChain to enable .cors(Customizer.withDefaults()) so Spring Security honours the CORS configuration instead of blocking cross-origin calls.
3. Restart the application and, from Postman, add an Origin: http://localhost:3000 header to a request to /api/secure/data to simulate a cross-origin browser call.
4. Verify the response includes the Access-Control-Allow-Origin header matching the allowed origin.
5. Repeat the same Origin-header test for POST /api/secure/data, PUT /api/secure/data/{id} and DELETE /api/secure/data/{id} to confirm the CORS configuration covers all four HTTP verbs, not only GET.
6. Change the Origin header to a disallowed value (e.g., http://evil.com) and confirm the request is blocked/CORS headers are absent.
7. Document the difference between a same-origin Postman call (no CORS headers needed) and a simulated cross-origin call (CORS headers required).

### Postman API Testing:

| Method | Endpoint (Postman) | Request Body / Params                                                      | Expected Response                                                                     |
|--------|--------------------|----------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| GET    | /api/secure/data   | Header: Origin: http://localhost:3000 + Basic Auth                         | 200 OK with Access-Control-Allow-Origin: http://localhost:3000 header present         |
| POST   | /api/secure/data   | Header: Origin: http://localhost:3000 + Basic Auth + { "text": "Note" }    | 201 Created with Access-Control-Allow-Origin header present                           |
| PUT    | /api/secure/data/1 | Header: Origin: http://localhost:3000 + Basic Auth + { "text": "Updated" } | 200 OK with Access-Control-Allow-Origin header present                                |
| DELETE | /api/secure/data/1 | Header: Origin: http://localhost:3000 + Basic Auth                         | 204 No Content with Access-Control-Allow-Origin header present                        |
| GET    | /api/secure/data   | Header: Origin: http://evil.com + Basic Auth                               | Response lacks Access-Control-Allow-Origin — blocked by CORS policy in a real browser |